

COL 7364/764 Assignment 1 - Relevance Feedback and Reranking

2026 - Version 1.0
(dated 19th September 2026)

Important Dates

This assignment runs in three phases.

Phase	Description	Date
Competition Round	Continually updated leaderboard which measures how robust your feedback models are to noisy initial rankings. The leaderboard is computed against a held out noise model on an held out benchmark (will be revealed to you only after this deadline). You may submit your implementation as many times as you want during this phase. Submissions will be on a submission website, accessible from within IITD network / VPN.	26th September 2026, 5PM IST – 7th October 2026, 5PM IST
Report Submission	You are required to submit a report that explains your approach both in the initial submission phase as well as the competition round. You are also required to submit all the performance metrics (see the document below) you had in the released test dataset and topics in each phase. Report to be submitted on Moodle.	9th October 2026, 11:59PM IST

Instructions

Follow all instructions. Submissions not following these instructions will not be evaluated.

1. This programming assignment is to be done by each student individually. Do not collaborate by sharing code, algorithm, and any other pertinent details with each other. You are free to discuss and post questions to seek clarifications. Please post all discussions related to this assignment with “hw1” tag on Piazza.
2. All programs have to be written either using Python or C/C++ programming languages only. The machine we use to evaluate your submissions is running a Ubuntu 24.04.3 LTS, with Python 3.12.3, 4 CPU cores, 8 GB RAM, GCC/G++ 13.3.0.
We recommend you to use same versions to avoid the use of deprecated/unsupported functions or take care to ensure required compatibility.
3. A single zip of the source code has to be submitted. The submitted zip file, when deflated, should result in the same directory structure as in the associated GitHub repository. Your submission will not get evaluated if any file / directory in the repository is missing.
The list of files to be submitted, along with their naming conventions and call signatures as given in the instructions below.

4. **Note that we will use only Ubuntu Linux machines to build and run your assignments.** So take care that your file names, paths, argument handling etc. are compatible.
5. You *should not* submit data files. If you are planning to use any other “special” library, please talk to the instructor first (or post on Piazza).
6. Note that your submission will be evaluated using larger held-out collection. All documents will be in English, have sentences terminated by a period, and all documents are in one single directory. The overall format of documents will be same as given in your training (as specified through `ir_datasets` interface).
7. **Note that there will be no deadline extensions. Apart from the usual “please start early” advise, I must warn you that this assignment requires significant amount of implementation effort, as well as some ‘manual’ tuning of parameters to get good speed up and performance. Do not wait till the end.**

The file structure in each submission must be exactly as given in the associated GitHub repository. Do not remove or rename files already given in the repository. You are free to add more files as needed.

1 Assignment Description

The goal of this assignment is to build a **robust** relevance-feedback retriever and survive a noise-contaminated pseudo-relevant set. For every query, you are given a top-K candidate pool — produced by an undisclosed first-pass retriever, offline, over the full corpus. You are expected to implement a language-model reranking score over that pool, and a relevance-model feedback layer (RM1/RM2/RM3) on top of it. You are not required to build an index.

Your submissions will be scored based not only on how well your feedback model improves ranking when the pseudo-relevant set is clean, but also on how much worse the reranking system gets when the pseudo-relevant set is quietly polluted with off-topic / nonrelevant documents before it is given to you. This is called **query drift**, and it is a well-documented failure mode of pseudo-relevance feedback: expansion terms mined from a pseudo-relevant set that isn’t actually all relevant can drag the query toward the wrong topic entirely [MSB98]. A submission that boasts a great $nDCG@10$ under ideal conditions, but fails dramatically when its input is realistically noisy is clearly not a desirable feedback system – it has essentially overfit. Such feedback implementations do not win in this assignment.

1.1 Learning Objectives

- Implement a unigram query-likelihood reranking score, with a smoothing method of your choice (Jelinek-Mercer or Dirichlet prior; see [ZL01] for why the choice matters more than intuition suggests), applied to a provided candidate pool rather than the whole corpus.
- Implement relevance-based language models for pseudo-relevance feedback — estimating a relevance model from a pseudo-relevant document set and using it to re-rank a candidate pool [LC01] — through the RM1 $\rightarrow$ RM2 $\rightarrow$ RM3 progression covered in lecture.
- Experience, hands-on, why pseudo-relevance feedback is a double-edged tool: it helps when the top-ranked pseudo-relevant set is genuinely on-topic and can actively hurt you when it isn’t (query drift).
- Understand a concrete failure mode of single-condition evaluation metrics: a clean-set $nDCG@10$ number does not tell you how a system behaves under realistic input noise, and a leaderboard that only ever measures the clean condition rewards exactly the systems that are most fragile to drift.

1.2 Background

1.2.1 Query Likelihood

The traditional query likelihood model scores a document, treating a language model estimated by the document generating the query. Ordinarily, computing this over an entire corpus and sorting is itself the retrieval step. But

in this assignment we will use this for **reranking** – i.e., score documents only for the documents in a candidate pool the harness has given you and return them re-sorted decreasing order of score. The candidate pool is often top-100 or top-1000 documents from an undisclosed first-pass retriever. The query likelihood model we covered in the class allows for two forms of smoothing – *Jelinek-Mercer* and *Dirichlet prior*. Zhai and Lafferty’s work has found that the two smoothing techniques behave differently across query and document lengths — Dirichlet tends to suit short, precise queries better and JM tends to suit long, verbose ones — which is itself a small, early example of this assignment’s theme: a single fixed setting tuned on one query distribution is not automatically the right choice on another.

Optional – beyond Unigram (Bigram). If you want to extend your correctly-implemented unigram baseline with a bigram or positional component (e.g. interpolating a bigram-smoothed model with your unigram one), that is fair game for competitive edge. Do not assume it is automatically more drift-resistant, though: bigram co-occurrence counts within a small pseudo-relevant set are sparser than unigram counts over the same documents, and sparser statistics are not obviously more robust to a contaminated set — they could just as easily be more sensitive to it. This is worth actually measuring, not assuming, if you go this route.

1.2.2 Relevance-based language models (RM1 / RM2 / RM3)

Lavrenko & Croft’s relevance models estimate a distribution $P(w|R)$ over the vocabulary that approximates the language of the documents relevant to the query, using only the top-scoring documents from an initial retrieval as pseudo-relevant evidence. In this assignment, this pseudo-relevant set is supplied to you directly as `pseudo_relevant_doc_ids` rather than being something you need to compute. Estimating the $P(w|R)$ is your job. You are expected to implement all the three, **RM1**, **RM2**, **RM3** variants that we studied.

1.2.3 Candidate Pool

Every retrieval call – `score_candidates()` and `relevance_model_feedback()` – in this assignment operates only on the candidate pool for the query given to you by the harness. This candidate pool is drawn from `data/candidates_dev.jsonl` (or a held-out equivalent during leaderboard / grading time). Given that this candidate pool is large enough (typically 100 or 1000), a strong reranker has a good chance to separate itself from a weak one. In fact, the toy dataset’s pool is deliberate full-recall to enable you to debug. The `score_candidates()` and `relevance_model_feedback()` may only return document ids from the pool they were given.

1.2.4 Query Drift

Every term in a pseudo-relevant document that is not genuinely on-topic is a term your relevance model has no way to distinguish from a genuinely useful one — it only sees documents, never ground-truth labels. When the pseudo-relevant set is mostly on-topic, feedback reliably helps; when it is contaminated with even a modest fraction of off-topic documents, an aggressive relevance model can pull the ranking toward the contamination instead of the query. A leaderboard that only evaluates the clean condition cannot distinguish a genuinely robust feedback-based reranker from an aggressive one that happens to get *lucky* on the specific pseudo-relevant sets the dev set produces. Both will have the same `nDCG@10`, but in the real-world setting where it is very likely that the first stage retrieval is polluted, only the robust one works.

2 Task Specification

2.1 What you must implement

You must implement the following from scratch – no existing IR library (Pyserini, Whoosh, `rank_bm25`, `gensim`’s LM utilities etc.) for scoring or language-model estimation. Please note that there will be no internet connectivity during the evaluation, so only some basic packages / libraries can be imported. You may reuse your own Assignment 1 code where it’s useful, but you must submit it as well.

Unigram Query-Likelihood Reranker (REQUIRED) , with a smoothing method of your choice, tunable smoothing parameter, exposed as `score_candidates()` entrypoint. It reranks the given candidate pool for the query (and only that pool).

Higher-order language models (optional) . `score_candidates()` and `relevance_model_feedback()` are graded purely on the ranked list they return, without inspecting the specific implementation used. If you would like to extend the correctly implemented Unigram model with bigram, positional, or proximity signals (some of which may need additional self-study), you are free to do so. *It is not to be treated as a substitute for correct unigram baseline.*

Relevance-model feedback (RM1 $\rightarrow$ RM3) , exposed as the required `relevance_model_feedback()` function. This function receives two document-ID lists as arguments – `pseudo_relevant_doc_ids` is the (possibly noise-perturbed) seed used *only to estimate the relevance model*, while `candidate_doc_ids` is the pool of documents that you actually rerank and return. `pseudo_relevant_doc_ids` is not necessarily related to what your own `score_candidates()` produced for that query, and it may not even a subset of `candidate_doc_ids`.

An adversarial pseudo-relevant set (Optional, bonus) A small JSON file of your own construction, mapping a handful of dev queries to document-ID lists you believe will result in maximal drift in an unknown opponent's feedback function when used as `pseudo_relevant_doc_ids`. It will be scored head-to-head against the rest of the class's submission.

2.2 What must you submit?

Your submission is the functions in `submission/feedback.py` –

- `prepare(corpus_path),`
- `score_candidates(query, candidate_doc_ids, k),` and
- `relvance_model_feedback(query, pseudo_relevant_doc_ids, candidate_doc_ids, k)`

(check the associated repository). **Optionally**, you also have to submit `submission/adversarial_set.json`.

For every query, the harness reads that query's candidate pool from a provided `candidates_dev.jsonl` file and passes it to you as `candidate_doc_ids`; *you never assemble this pool yourself and never see documents outside it*. Note that unlike Assignment 1, there is no build/load process-isolation step and no Docker image: the harness imports your module directly and calls these functions in-process.

2.3 Interface and Conformance

As mentioned above, only functions your submission exposes to the evaluation harness are those listed in the `submission/feedback.py`. The associated repository contains a trivial, fully-working baseline that runs end-to-end through the harness; fork it, keep the same shape, replace the internals.

Enforced wall-clock timeout and inter-submission time Because grading is in-process rather than containerised this time, `prepare()` and each retrieval call are subject to a generous but enforced wall-clock cap (stated in `docs/GRADING.md`) since every call is bounded by the candidate pool size (100s of documents), not corpus size; exceeding it on a query is scored as a miss on that query, not a harness crash. Similarly, to prevent over-loading the evaluation servers, there will be a strict time-delay between two consecutive submissions by the student. Specific delay will be adaptive and will be visible on the submission site.

Strict requirements on what is allowed and what is not. You may only return document IDs from the candidate pool that you are given. Your `relevance_model_feedback()` **must not** assume a previous call to `score_candidates()`. The harness may call it directly with harness constructed document ID list, independent of whatever was computed/returned by `score_candidates()`.

3 Dataset and Tools

Corpus: the same toy set as Assignment 1 for local development (`data/toy/`, 20 documents, 6 queries, graded qrels) plus the real assignment corpus via `scripts/download_full_corpus.py` — same format, same code path, no changes needed between toy and real. You are never required to load or scan this at query time; `prepare()` may use it to estimate collection-level background statistics $P(w | C)$, which is legitimate one-time language-model setup, not the retrieval step itself.

Candidate pool (`candidates_dev.jsonl`) : released alongside the dev topics, one entry per query — a list of (`doc_id`, `score`) pairs from the undisclosed first-pass retriever, best-first, read via `harness/candidates_io.py`. The toy set’s pool is deliberately full-recall (every judged-relevant document for every toy query is inside it) so it stays a reliable debugging tool; the real dev pool is a realistic top-100 and will not have that guarantee for every query. The held-out grading pool is never released, same as Assignment 1’s held-out topic set.

Dev topics + qrels: released for local tuning, same as Assignment 1.

Practice noise recipe (public): `harness/noise_injection.py` implements a fully documented noise-injection function you can run locally — swap a stated fraction of a pseudo-relevant set for documents sampled from elsewhere in the same candidate pool, at levels you choose — so you can test your own robustness before submitting. Sampling noise from within the pool rather than from the whole corpus is deliberate: it models “plausible but wrong” contamination (a document the first-pass retriever also considered plausible, but that isn’t actually relevant), which is harder to reject than wildly unrelated noise. This is for your own experimentation; it is not the recipe used for grading (see below).

Grading noise recipe (undisclosed): the exact noise fractions, sampling strategy, and number of perturbed conditions used to compute your real score are not released, for the same reason Assignment 1’s held-out topics and baseline parameters weren’t: a submission tuned against a known perturbation recipe is measuring its ability to guess the recipe, not its robustness. Build against the public practice recipe; do not assume the real one matches it exactly.

4 Competition Mechanics and Scoring

Format: open leaderboard, unlimited submissions before the deadline, public dev-topic scores visible in real time, private held-out-topic scores revealed at the deadline. Every accuracy-bearing component below is percentile-ranked against the whole class before weighting, not scored on raw metric value.

Rank component	Metric	Weight
Base LM accuracy	nDCG@10 / MAP@10 of <code>score_candidates()</code> reranking the provided candidate pool, percentile-ranked against the class	30%
Clean feedback accuracy	nDCG@10 of <code>relevance_model_feedback()</code> given the harness’s clean (0%-noise) pseudo-relevant set, percentile-ranked	25%
Drift robustness	Retention: the percentile rank of (average nDCG@10 across the undisclosed noisy conditions) $\div$ (your own Track B clean-condition nDCG@10) — i.e. ranked on how little you degrade, not on raw noisy-condition accuracy alone	35%
(Bonus, Optional) Adversarial	Your submitted adversarial set’s average drift induced in the rest of the class, plus your own resistance to everyone else’s adversarial sets — additive only, cannot push you past 100%	$\pm 10\%$

A note on the bonus track: the round-robin is run once, by TAs, after the deadline, across every submitted `adversarial_set.json` against every submitted `relevance_model_feedback()` (excluding self-pairs). It is genuinely optional — a submission that skips it is simply not eligible for the bonus, with no penalty.

5 Oral Defense

After the private leaderboard closes, a random selection of students sit a 10–15 minute oral defense: a walkthrough of one design decision, 2–3 targeted questions on your own code, and a live perturbation. If the team cannot explain or meaningfully modify their own submission, the leaderboard-performance component below is capped at 50% of its earned value regardless of rank.

6 What is to be submitted?

After the live competition closes, the following have to be submitted:

1. A short report (2–3 pages): a plot or table of $\text{nDCG}@10$ vs. λ on the dev set at 0% noise, the same sweep at the highest noise level your local `noise_injection.py` practice recipe supports, and a short discussion of the tradeoff you observed between expansion strength and drift resistance.
2. A one-paragraph description of your final λ and smoothing choice and why you settled there — specifically, what you traded off to get there.
3. An AI-use disclosure statement and a code provenance statement.

Note that your submissions will be stored on the evaluation site, so you need not submit again.

7 Grading policy

Component	Weight	Criteria
Leaderboard performance	40%	The leaderboard score, scaled by percentile rank among the class on the private held-out set; clearing the undisclosed baseline is a pass/fail floor. Capped at 50% of earned value if the understanding check indicates the team cannot explain their own submission.
Understanding check	15%	You correctly explain λ /smoothing choices and correctly predict the qualitative effect of a live, examiner-chosen noise perturbation.
Correctness of required components	20%	Your required unigram <code>score_candidates()</code> and RM1/2/3-based <code>relevance_model_feedback()</code> are both correctly implemented and independently verifiable against small hand-checkable examples, not leaderboard score alone.
Report quality	15%	The λ -vs-noise tradeoff analysis is specific and evidenced (real plots from your own runs), not generic or asserted without data.

8 Academic Integrity

No existing search/indexing library may be used for the required components. Pretrained language models or embeddings are out of scope for this assignment — that machinery is introduced later in the course. Discussing high-level strategy with classmates is fine; sharing code or a tuned parameter file is not.

LLM assistance is permitted but must be declared in an AI-use disclosure statement, and any external code consulted (including Pyserini source, public repos, or prior submissions) must be listed in a code provenance statement with the specific delta you implemented. Course staff run similarity detection across current- and prior-term submissions and public repositories; flagged submissions receive closer scrutiny at the oral defense rather than an automatic penalty. A team that cannot explain code flagged as similar to an external source, in the defense, will be treated as an integrity case, not merely graded down.

References

- [LC01] Victor Lavrenko and Bruce Croft. Relevance Based Language Models. In *SIGIR*, 2001.
- [MSB98] Mandar Mitra, Amit Singhal, and Chris Buckley. Improving Automatic Query Expansion. In *SIGIR*, 1998.
- [ZL01] ChengXiang Zhai and John Lafferty. A Study of Smoothing Methods for Language Models applied to Ad-hoc Information Retrieval. In *SIGIR*, 2001.